ChatGPT

SQL Query Optimization Interview Questions (MySQL)

Multiple Choice Questions (50)

1. **Which of the following is a primary benefit of adding an index to a column used in a WHERE clause?**
2. A. Faster SELECT query filtering and retrieval.
3. B. Faster INSERT operations.
4. C. Reduced disk usage.
5. D. None of the above.

Answer: A.
Explanation: Indexes speed up queries by allowing the database to filter rows more efficiently in the `WHERE` clause [1]. They significantly reduce the need for full table scans, speeding up data retrieval.

1. **Why is using `SELECT *` generally discouraged in query optimization?**
2. A. It fetches unnecessary columns, increasing I/O and slowing performance.
3. B. It speeds up query execution.
4. C. It simplifies the query plan for the optimizer.
5. D. It automatically creates indexes.

Answer: A.
Explanation: `SELECT *` retrieves all columns, including those not needed, which increases the amount of data scanned and transferred. Selecting only the required columns keeps the query leaner and often improves performance [2].

1. **What is the purpose of the `EXPLAIN` statement in MySQL?**
2. A. It executes the query and returns results with extra details.
3. B. It shows the execution plan MySQL will use for a given query.
4. C. It creates a query template for debugging.
5. D. It drops unused indexes.

Answer: B.
Explanation: Prepending `EXPLAIN` to a `SELECT` query causes MySQL to output the execution plan. This includes which indexes are used and how tables are joined, helping developers understand and optimize query performance [3].

1. **Between JOINs and subqueries, which generally executes faster in MySQL?**
2. A. JOINs, because they often avoid redundant processing and use indexes efficiently.
3. B. Subqueries, because they break the problem into smaller queries.
4. C. No difference in execution speed.
5. D. Depends only on table size.

Answer: A.
Explanation: Joins typically outperform semantically equivalent subqueries in MySQL. They combine tables in one pass and make better use of indexes, whereas subqueries (especially correlated ones) may be executed repeatedly [4].

1. **Which part of a SELECT query would most benefit from having an index on the involved columns?**
2. A. SELECT clause.
3. B. WHERE clause.
4. C. ORDER BY clause.
5. D. GROUP BY clause.

Answer: B.
Explanation: Columns used in the `WHERE` clause benefit most from indexing. MySQL can then quickly locate matching rows without scanning the entire table [1].

1. **What is the effect of using a `LIMIT` clause in a SELECT query?**
2. A. Limits the number of columns retrieved.
3. B. Limits the number of rows returned.
4. C. Randomizes the query results.
5. D. Expands the result set.

Answer: B.
Explanation: The `LIMIT` clause restricts the result set to the specified number of rows. This avoids fetching unnecessary rows and can greatly reduce data processing when only a subset is needed [5].

1. **Why can adding an `ORDER BY` clause slow down a query on a large table?**
2. A. It prevents index use.
3. B. The database must sort all qualifying rows before returning results.
4. C. It converts the query into a full table scan.
5. D. ORDER BY is not supported in MySQL.

Answer: B.
Explanation: An `ORDER BY` causes MySQL to sort all matching rows. Without an appropriate index, this sort step (often using filesort) can be very time-consuming on large tables [6].

1. **How can an `ORDER BY ... LIMIT ...` query be optimized for performance?**
2. A. By avoiding any indexes on the ordering columns.
3. B. By creating an index on the columns used in `ORDER BY`.
4. C. By using `UNION ALL` instead of `ORDER BY`.
5. D. By turning off MySQL caching.

Answer: B.
Explanation: If the `ORDER BY` column is indexed, MySQL can retrieve rows in sorted order directly from the index, avoiding an extra sort. This makes applying the `LIMIT` much faster [7].

1. **In an `EXPLAIN` output, a type of `ALL` indicates a "Full Table Scan." Which scenario commonly leads to this?**

2. A. Query on a tiny table (e.g., < 10 rows).
3. B. No usable index exists for the `WHERE` conditions.
4. C. Using an indexed column in the `WHERE` clause.
5. D. Using a temporary table.

Answer: B.
Explanation: `ALL` means MySQL is scanning the whole table. This usually happens when there is no applicable index on the columns used in the `WHERE` clause [8], forcing a full scan of the table.

1. **Which MySQL statement can be used to see which indexes a `SELECT` query will use?**

    - A. EXPLAIN SELECT ...
    - B. SHOW INDEX FROM ...
    - C. ANALYZE TABLE ...
    - D. DESCRIBE SELECT ...

    Answer: A.
    Explanation: Using `EXPLAIN` before a query displays details about the execution plan, including which indexes (if any) MySQL chooses to use for each table in the query [9].

2. **In MySQL, declaring a column as `PRIMARY KEY` has which of the following effects?**

    - A. Creates a unique index on that column automatically.
    - B. Disallows any other indexes on the table.
    - C. Slows down lookups on that column.
    - D. Allows duplicate NULL values.

    Answer: A.
    Explanation: A primary key in MySQL is a unique index on that column (or set of columns) [10]. It guarantees uniqueness and not-null on the key column(s), and speeds up lookups using that key.

3. **Given a composite index on `(country, state)`, which query will NOT benefit from this index?**

    - A. `WHERE country = 'US'`
    - B. `WHERE country = 'US' AND state = 'CA'`
    - C. `WHERE state = 'CA'`
    - D. `WHERE country = 'US' AND state LIKE 'C%'`

    Answer: C.
    Explanation: MySQL uses composite indexes left-to-right. A condition on `state` alone (without `country`) cannot use the `(country,state)` index because `country` (the leftmost column) is not restricted [11].

4. **What is a typical trade-off when denormalizing a database for performance?**

    - A. It increases storage usage but can speed up read queries.
    - B. It decreases query speed but reduces disk space.

- C. It allows infinite rows without performance loss.
- D. It always improves performance without downsides.

Answer: A.
Explanation: Denormalization often duplicates data (using more storage) so that reads (which avoid expensive joins) are faster. The trade-off is extra disk space and more complex updates when data changes [12].

5. **Why should you avoid using functions on indexed columns in the `WHERE` clause (e.g., `WHERE YEAR(date)=2020`)?**

   - A. Functions execute faster than comparisons.
   - B. Functions change the data type, causing errors.
   - C. Applying a function prevents using the index, leading to full table scans.
   - D. MySQL does not allow functions in `WHERE`.

   Answer: C.
   Explanation: When you wrap a column in a function (like `YEAR(date)`), MySQL cannot use the index on that column, since the index stores the raw column values [13]. The function must be applied to every row at runtime, which disables index usage and slows the query.

6. **Which practice can improve query performance by reducing data processing?**

   - A. Using `SELECT DISTINCT *`.
   - B. Selecting only the needed columns instead of `SELECT *`.
   - C. Removing the `WHERE` clause.
   - D. Including all columns in `GROUP BY`.

   Answer: B.
   Explanation: Fetching only the necessary columns means the database reads and transfers less data. This makes queries more efficient, especially on wide tables, because it avoids unnecessary I/O [2].

7. **For best performance, which columns should you use to join two tables?**

   - A. Unindexed or arbitrary columns.
   - B. Columns with similar names but no relationship.
   - C. Columns designated as primary or foreign keys.
   - D. Randomly chosen columns.

   Answer: C.
   Explanation: Joins should be on primary/foreign key columns whenever possible. These are typically indexed and ensure the join matches the intended logical relationship, preventing needless scans [14].

8. **Suppose a table `t1` has an index on `(c1,c2,c3)`. Which `GROUP BY` clause can use this index for faster grouping?**

   - A. `GROUP BY c1, c2`
   - B. `GROUP BY c2, c3`
   - C. `GROUP BY c3, c1`
   - D. `GROUP BY c1, c3`

   Answer: A.
   Explanation: MySQL can use an index for `GROUP BY` only if the grouped columns are a leftmost prefix of the index. `(c1,c2)` is a leftmost prefix of `(c1,c2,c3)`, so grouping by `c1, c2` can leverage the index [15].

9. **Why might `UNION ALL` perform better than `UNION`?**

   - A. `UNION ALL` removes duplicates, which `UNION` does not.
   - B. `UNION` is faster because it avoids distinct processing.
   - C. `UNION ALL` retains duplicates and avoids the extra work of deduplication.
   - D. There is no difference.

   Answer: C.
   Explanation: `UNION` must remove duplicate rows (using sorting or hashing), which adds overhead. `UNION ALL` simply concatenates results without deduplication, so it typically runs faster and uses fewer resources [16].

10. **Which construct can often be faster when checking for existence of matching rows in another table?**

    - A. `EXISTS`, because it stops at the first match.
    - B. `IN`, because it precomputes values.
    - C. They are always identical.
    - D. Neither; always use subqueries.

    Answer: A.
    Explanation: An `EXISTS` subquery is usually faster for large datasets since it stops once a matching row is found. In contrast, `IN` would evaluate all values returned by the subquery, which can be slower [17].

11. **If a query has `WHERE region = 'East' OR region = 'West'`, what optimization technique is recommended?**

    - A. Rewrite using `UNION ALL` with separate queries.
    - B. Always use `OR`; it is most efficient.
    - C. Use a full table scan.
    - D. Add an index on `region` to automatically handle OR.

Answer: A.
Explanation: OR conditions, especially across multiple values or tables, can be slow. Splitting the query into separate parts joined by `UNION ALL` can perform better, as MySQL can then use indexes independently for each part [18].

12. **What is the purpose of running `ANALYZE TABLE tbl_name` in MySQL?**

    - A. To clear table data.
    - B. To update table statistics for the optimizer.
    - C. To create a backup of the table.
    - D. To add an index automatically.

    Answer: B.
    Explanation: `ANALYZE TABLE` updates index statistics, giving the optimizer accurate information on data distribution. This helps MySQL choose efficient execution plans and avoid unnecessary full scans [19].

13. **What does the `FORCE INDEX (index_list)` clause do in a MySQL query?**

    - A. It tells MySQL to use only the given indexes and ignore others.
    - B. It tells MySQL to use the specified index(es) even if it might not choose them by default.
    - C. It forces all indexes to be dropped after the query.
    - D. It is not a valid MySQL clause.

    Answer: B.
    Explanation: `FORCE INDEX` instructs MySQL to prefer the listed index(es) for the query, even if the optimizer thinks a table scan would be faster. This can be used to override the optimizer's choice and avoid full table scans [20].

14. **Which clause inherently removes duplicate rows from the result set?**

    - A. `GROUP BY`
    - B. `DISTINCT`
    - C. `LIMIT`
    - D. `JOIN`

    Answer: B.
    Explanation: The `DISTINCT` keyword eliminates duplicate rows in the result. (Note: `GROUP BY` groups rows and can have a similar effect in some cases, but `DISTINCT` explicitly removes duplicates.)

15. **What is a self-join?**

    - A. A join of a table with itself.
    - B. A join with no ON condition.
    - C. A cross join.
    - D. A join that always returns zero rows.

Answer: A.
Explanation: A self-join is when a table is joined with itself, typically by aliasing it as two different tables in the query. This allows comparing rows within the same table.

16. **Using `LIMIT 1000 OFFSET 0` vs `LIMIT 1000 OFFSET 1000`, what is generally true?**

    - A. Both have identical performance.
    - B. The one with offset 1000 reads and discards the first 1000 rows.
    - C. OFFSET has no effect on performance.
    - D. Offset always makes query faster.

    Answer: B.
    Explanation: When using an offset, MySQL must skip (read and discard) rows up to the offset. `OFFSET 1000` causes the database to walk through the first 1000 rows before fetching the next 1000, which can slow down queries with large offsets.

17. **What is a covering index?**

    - A. An index on all columns of a table.
    - B. An index that includes all columns needed by a query, so no table access is needed.
    - C. An index that covers only text columns.
    - D. An index created by the optimizer automatically.

    Answer: B.
    Explanation: A covering index is one that contains every column referenced in a query (SELECT, WHERE, etc.). When a covering index is used, MySQL can fulfill the query using only the index without reading the full table, improving performance.

18. **In queries, using table aliases (e.g., `FROM employees e`) is primarily for:**

    - A. Hiding table names from the database.
    - B. Making queries shorter and easier to read, especially with multiple tables.
    - C. Improving query performance automatically.
    - D. Creating temporary copies of tables.

    Answer: B.
    Explanation: Table aliases are a convenience for writing queries. They make column references shorter and improve readability, especially in complex joins. They do not directly affect query performance.

19. **Does `JOIN` order affect the result of a query?**

    - A. Yes, the left table must be smaller.
    - B. No, JOIN order can affect performance but not result (with INNER JOIN).
    - C. Yes, it changes the result set.
    - D. No, JOIN order is irrelevant in all cases.

Answer: B.
Explanation: For INNER JOINs, the order of tables does not change the result, though it can influence which table MySQL reads first (possibly affecting performance). For OUTER JOINs, left vs right join does change which side is preserved.

20. **What does `GROUP BY` do in a query?**

    - A. Filters rows before aggregation.
    - B. Groups rows with identical values in specified columns into summary rows.
    - C. Orders rows by group.
    - D. Removes duplicate rows.

    Answer: B.
    Explanation: `GROUP BY` clusters rows that share the same values in the listed columns. It is commonly used with aggregate functions (e.g., `COUNT`, `SUM`) to produce a summary per group.

21. **Why should you avoid using `ORDER BY RAND()` on large tables?**

    - A. Because it returns no results.
    - B. Because it causes a full sort of all rows, which is very slow.
    - C. Because MySQL doesn't support `RAND()`.
    - D. Because it creates a temporary table of all data.

    Answer: B.
    Explanation: `ORDER BY RAND()` assigns a random value to each row and sorts the entire result set by that value. On large tables this is very expensive. Alternative methods (like selecting a random ID) are recommended for efficiency.

22. **What is the effect of using `UNION` instead of `UNION ALL` when not needing duplicate elimination?**

    - A. No effect, they perform identically.
    - B. `UNION` will be slower because it removes duplicates.
    - C. `UNION` will be faster because it uses one less step.
    - D. `UNION ALL` is slower because it returns duplicates.

    Answer: B.
    Explanation: `UNION` removes duplicate rows, which requires an extra sorting or hashing step. If you don’t need to remove duplicates, using `UNION ALL` avoids that work and is faster [16].

23. **Which clause can help avoid scanning the entire table by indicating how many rows to read?**

    - A. `HAVING`.
    - B. `LIMIT`.
    - C. `JOIN`.
    - D. `FOR UPDATE`.

Answer: B.
Explanation: The `LIMIT` clause tells MySQL to retrieve only a specified number of rows. This can prevent unnecessary scanning of all rows if only a subset is needed [5].

24. **What does adding an appropriate index do for `ORDER BY` or `GROUP BY` columns?**

    - A. It has no effect on sorting/grouping performance.
    - B. It may allow MySQL to retrieve rows in order directly from the index, avoiding extra sort.
    - C. It duplicates the data in those columns.
    - D. It changes the table's default sort order permanently.

    Answer: B.
    Explanation: An index on `ORDER BY` or `GROUP BY` columns means MySQL can access the data in sorted order without a separate sort step. This is especially beneficial for large datasets (see GROUP BY index use [15]).

25. **Which is generally faster for filtering a column against a list of values, `IN` or multiple `OR` conditions?**

    - A. `IN` is generally faster or equivalent, as it simplifies the expression.
    - B. Multiple `OR` is always faster.
    - C. They are always identical in performance.
    - D. Neither; always use subqueries.

    Answer: A.
    Explanation: Using `IN (val1, val2, ...)` is functionally similar to `column = val1 OR column = val2`, but can be clearer and better optimized. MySQL can often handle `IN` efficiently, whereas complex OR expressions may be slower [18].

26. **What does the `USING INDEX` or `USING INDEX FOR GROUP-BY` in an `EXPLAIN` plan indicate?**

    - A. That the query is not using any index.
    - B. That MySQL is retrieving data solely from the index (covering index) or using the index for grouping.
    - C. That a full table scan is happening.
    - D. That the index is corrupted.

    Answer: B.
    Explanation: `Using index` (in `Extra`) means the query used a covering index (no table lookup needed). `Using index for group-by` indicates that grouping was done via the index order without creating a temp table [15].

27. **Which scenario can cause the optimizer to ignore an index and do a full table scan?**

    - A. When the table is extremely small.
    - B. When the indexed column has too many distinct values.
    - C. When the index is exactly on the queried column.

- D. When using `ORDER BY` with an index.

Answer: A.
Explanation: For very small tables, MySQL may choose a table scan as it can be faster than using an index. (Also, if a WHERE condition covers most rows or uses low-cardinality keys, a scan may be chosen.)

28. **What is the effect of having many unused columns selected (e.g., `SELECT *`) when an index only covers some columns?**

    - A. No effect, since only indexed columns are accessed.
    - B. It forces MySQL to read the full rows from disk after using the index (bookmark lookup), incurring extra I/O.
    - C. It automatically drops those columns.
    - D. It duplicates the data in memory.

    Answer: B.
    Explanation: Even if an index is used to find rows, selecting non-indexed columns (`SELECT *`) means MySQL must retrieve the rest of the row from the table (using the index row pointer). This extra lookup slows the query.

29. **What is a common drawback of using a `SELECT DISTINCT` on a large dataset?**

    - A. It is always faster than `GROUP BY`.
    - B. It requires sorting or hashing to remove duplicates, which can be slow.
    - C. It duplicates all rows.
    - D. It ignores indexes entirely.

    Answer: B.
    Explanation: `SELECT DISTINCT` forces MySQL to eliminate duplicates, usually by sorting or building a hash of values. On large datasets, this can be resource-intensive.

30. **Which optimization can help when a query uses a `LIKE 'prefix%'` condition?**

    - A. Adding an index on the column.
    - B. Converting `LIKE` to `REGEXP`.
    - C. Using a subquery.
    - D. Removing the wildcard.

    Answer: A.
    Explanation: A leading-anchored `LIKE 'prefix%'` can use a standard index on the column (because it matches from the start). Ensuring an index on that column greatly speeds such searches. (By contrast, `'%suffix'` cannot use an index.)

31. **Why is `ORDER BY` on multiple columns (e.g., `ORDER BY a, b`) slower than on one column?**

    - A. MySQL cannot sort on multiple columns.
    - B. It requires sorting by the first column, then the second for ties, which is more work.

- C. It automatically drops duplicates.
- D. It ignores the first column.

Answer: B.
Explanation: Sorting by multiple columns typically requires a more complex sort. If no suitable composite index exists on those columns in order, MySQL must do an explicit sort on both columns, which is slower than sorting by one column.

32. **What does setting `SQL_CALC_FOUND_ROWS` do when combined with `LIMIT`?**

    - A. It is required for `LIMIT` to work.
    - B. It forces MySQL to count the total matching rows even with `LIMIT`, to allow retrieval via `FOUND_ROWS()`.
    - C. It optimizes the query to run faster with `LIMIT`.
    - D. It disables the `LIMIT`.

    Answer: B.
    Explanation: `SQL_CALC_FOUND_ROWS` tells MySQL to calculate how many rows *would* be returned without `LIMIT`. After a `SELECT ... LIMIT ...`, you can run `SELECT FOUND_ROWS()` to get that total. This incurs extra work, so use only if needed.

33. **Why might you prefer `UNION ALL` over multiple separate `SELECT` statements with `LIMIT`?**

    - A. It combines queries into one, allowing the optimizer to handle them together.
    - B. It forces the use of an index.
    - C. It always runs faster because it avoids internal table creation.
    - D. There is no advantage.

    Answer: A.
    Explanation: Using `UNION ALL` to combine multiple queries can be more efficient than running them separately in application code, since MySQL handles them in one execution. (However, performance depends on context.)

34. **Which of the following will generally be faster in MySQL?**

    - A. A JOIN that returns extra columns and rows.
    - B. The same JOIN with a `WHERE` clause limiting results early.
    - C. Removing the join condition to reduce the result.
    - D. Using SELECT * on all tables.

    Answer: B.
    Explanation: Filtering rows earlier with a `WHERE` clause can reduce the amount of data MySQL must process in the join, speeding up the query.

35. **What is the advantage of breaking a large query into smaller pieces (e.g., using temporary tables)?**

    - A. MySQL can automatically optimize across pieces.

- B. It may allow indexing intermediate results and reduce memory usage.
- C. It always runs slower.
- D. It avoids the need to write SQL.

Answer: B.
Explanation: Storing intermediate results can sometimes allow adding indexes on them or processing smaller result sets in each step, which may reduce overall work. This is context-dependent.

36. **What impact can selecting columns in a different order than the index have?**

- A. None, column order in SELECT doesn't matter.
- B. If the index is composite, the order of SELECT columns doesn't affect index use.
- C. It can break the index usage unless a covering index matches the SELECT list.
- D. It forces a table scan.

Answer: B.
Explanation: The order of columns in the SELECT clause does not affect which indexes are used; index usage is determined by WHERE, JOIN, and ORDER BY/GROUP BY clauses. Column order in SELECT only affects the order of output, not performance.

37. **Why should you avoid unnecessary calculations in the WHERE clause?**

- A. Calculations are always faster than comparisons.
- B. MySQL can't cache results with calculations.
- C. Calculations on columns (e.g., `col+1=5`) prevent using indexes, forcing full scans.
- D. Calculations cause syntax errors in WHERE.

Answer: C.
Explanation: Similar to using functions, performing calculations on columns in `WHERE` prevents MySQL from using indexes on those columns, because the raw indexed values no longer match. This can degrade performance.

38. **What does `GROUP_CONCAT` do and how can it affect performance?**

- A. It concatenates values into a string; it can increase memory usage for large groups.
- B. It groups rows automatically.
- C. It is an alias for `CONCAT`.
- D. It always improves performance by reducing rows.

Answer: A.
Explanation: `GROUP_CONCAT` aggregates values from multiple rows into a single string. For very large groups, the resulting string can become large, so MySQL may need more memory (controlled by `group_concat_max_len`), which can affect performance.

39. **Which type of JOIN returns only matching rows from both tables?**

- A. INNER JOIN

- B. LEFT JOIN
- C. RIGHT JOIN
- D. FULL OUTER JOIN

Answer: A.
Explanation: An `INNER JOIN` returns only rows where the join condition matches in both tables. This is both correct result-wise and generally the most efficient join to use when only matching data is needed.

40. **Why might `ORDER BY RAND()` not benefit from an index?**

    - A. It randomly orders rows, which cannot be precomputed in an index.
    - B. It always uses a full table scan by definition.
    - C. RAND() is non-deterministic and indexes require sorted order.
    - D. It doesn't affect index use.

    Answer: A.
    Explanation: Since `RAND()` assigns a random value to each row at query time, there's no fixed sort order. An index cannot predict this random order, so MySQL must assign random values and sort all rows at runtime, making it index-inefficient.

41. **What optimization helps queries that count rows (e.g., `SELECT COUNT(*)`) on large tables?**

    - A. Always adding DISTINCT.
    - B. Maintaining a counter or using indexed columns (e.g., a small auxiliary table or summary).
    - C. Using `ORDER BY`.
    - D. There is no way to speed up COUNT(*) queries.

    Answer: B.
    Explanation: Counting rows may involve scanning an index (like the primary key). Sometimes performance can be improved by maintaining a separate counter or using summary tables (especially if exact real-time count isn't needed on every query).

Optimize This Query (50)

1. **Inefficient Query:**

```sql
SELECT *
FROM employees
WHERE department = 'Sales';
```

Optimized Query:

```sql
SELECT id, name, salary
FROM employees
WHERE department = 'Sales';
```

Explanation: Selecting only the needed columns (`id, name, salary`) instead of `*` reduces the amount of data read from disk. This decreases I/O and speeds up the query [2].

2. **Inefficient Query:**

```sql
SELECT total_amount
FROM orders
WHERE customer_id IN (
    SELECT id FROM customers WHERE active = 1
);
```

Optimized Query:

```sql
SELECT o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE c.active = 1;
```

Explanation: Converting the subquery to an `INNER JOIN` lets MySQL use indexes on `customer_id` and `id`, so it doesn't have to execute the inner query for each row. This usually runs much faster [21].

3. **Inefficient Query:**

```sql
SELECT *
FROM products
WHERE category = 'Electronics' OR category = 'Books';
```

Optimized Query:

```sql
SELECT *
FROM products
WHERE category IN ('Electronics', 'Books');
```

Explanation: Using `IN` instead of multiple `OR` conditions simplifies the query. MySQL can optimize an `IN` list effectively, and it is functionally equivalent. This often leads to a more efficient plan (the optimizer treats it similarly to separate OR conditions) [18].

4. **Inefficient Query:**

```
SELECT name
FROM customers
ORDER BY registration_date;
```

Optimized Query:

```
ALTER TABLE customers ADD INDEX idx_reg_date (registration_date);
SELECT name
FROM customers
ORDER BY registration_date;
```

Explanation: Without an index on `registration_date`, MySQL must sort all customer rows by date. Adding an index allows it to retrieve rows in order, avoiding an expensive sort [7].

5. **Inefficient Query:**

```
SELECT *
FROM sales
WHERE YEAR(order_date) = 2021;
```

Optimized Query:

```
SELECT *
FROM sales
WHERE order_date BETWEEN '2021-01-01' AND '2021-12-31';
```

Explanation: The function `YEAR(order_date)` prevents use of an index on `order_date`. By rewriting it as a range on `order_date`, MySQL can use any index on that column, avoiding a full scan [13].

6. **Inefficient Query:**

```
SELECT DISTINCT user_id
FROM logins;
```

Optimized Query:

```sql
SELECT user_id
FROM logins
GROUP BY user_id;
```

Explanation: Both `DISTINCT` and `GROUP BY` can eliminate duplicates. MySQL often handles them similarly. Using `GROUP BY` can sometimes take advantage of an index on `user_id` (if present), producing the same result more efficiently.

7. **Inefficient Query:**

```sql
SELECT product_id
FROM inventory
UNION
SELECT product_id
FROM orders;
```

Optimized Query:

```sql
SELECT product_id
FROM inventory
UNION ALL
SELECT product_id
FROM orders;
```

Explanation: `UNION` removes duplicates, which requires extra sorting work. Since we used `UNION ALL`, duplicate elimination is skipped, making the query faster [16].

8. **Inefficient Query:**

```sql
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.id = d.manager_id;
```

Optimized Query:

```sql
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

Explanation: The original join condition was incorrect, causing a less efficient join. By joining on `department_id = d.id` (the proper foreign key relationship), the optimizer can use indexes correctly and reduce the result set to relevant matches.

9. **Inefficient Query:**

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
```

Optimized Query:

```
SELECT DISTINCT c.name
FROM customers c
JOIN orders o ON c.id = o.customer_id;
```

Explanation: If the intention is to get customers with orders, an `INNER JOIN` is more appropriate than a `LEFT JOIN` (which includes customers with no orders). Using `DISTINCT` ensures each name appears once. This reduces unnecessary row combinations and speeds up the query.

10. **Inefficient Query:**

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

Optimized Query:

```
ALTER TABLE employees ADD INDEX idx_salary (salary);
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 1;
```

Explanation: Without an index, sorting by `salary` is slow. Adding an index on `salary` lets MySQL find the top-salary row directly through the index. This greatly speeds up queries that order by `salary` with a LIMIT.

11. **Inefficient Query:**

```
SELECT city, COUNT(*)
FROM customers
GROUP BY city;
```

Optimized Query:

```
ALTER TABLE customers ADD INDEX idx_city (city);
SELECT city, COUNT(*)
FROM customers
GROUP BY city;
```

Explanation: Adding an index on `city` can allow MySQL to use the index for grouping, avoiding a large temporary table. Grouping on an indexed column is faster because it can iterate in order [15].

12. **Inefficient Query:**

```
SELECT *
FROM products
WHERE name LIKE '%phone';
```

Optimized Query:

```
SELECT *
FROM products
WHERE name LIKE 'Smart%phone';
```

Explanation: A leading wildcard (`%phone`) means no index can be used. By changing the pattern so it doesn't start with `%` (e.g., `'Smart%phone'`), an index on `name` (if it exists) can be utilized. If an exact suffix search is needed, consider full-text indexes.

13. **Inefficient Query:**

```
SELECT SUM(amount)
FROM sales;
```

Optimized Query:

```
ALTER TABLE sales ADD INDEX idx_amount (amount);
SELECT SUM(amount)
FROM sales;
```

Explanation: While summing all rows still requires scanning them, an index on `amount` may allow the database to read less data if there are conditions (not shown here). More effectively, maintaining a running total or a summarized table could avoid scanning all rows each time.

14. **Inefficient Query:**

```sql
SELECT *
FROM employees
WHERE department = 'HR'
LIMIT 10000;
```

Optimized Query:

```sql
SELECT *
FROM employees
WHERE department = 'HR'
LIMIT 100;
```

Explanation: If the application only needs 100 rows, asking for 10,000 is wasteful. Reducing the `LIMIT` to the actual needed number avoids unnecessary I/O and processing, improving performance.

15. **Inefficient Query:**

```sql
SELECT *
FROM employees e, departments d;
```

Optimized Query:

```sql
SELECT *
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

Explanation: The original query produces a Cartesian product (every employee with every department). Adding the proper `JOIN` condition (`e.department_id = d.id`) restricts the result to matching rows and greatly reduces processing.

16. **Inefficient Query:**

```sql
SELECT DISTINCT customer_id
FROM orders;
```

Optimized Query:

```sql
SELECT customer_id
FROM orders
GROUP BY customer_id;
```

Explanation: Replacing `DISTINCT` with `GROUP BY` can sometimes allow index usage on `customer_id`, since MySQL groups on that column. Functionally they are similar, but `GROUP BY` with an index can be faster.

17. **Inefficient Query:**

```
SELECT *
FROM orders
WHERE price = 100 OR price = 200;
```

Optimized Query:

```
SELECT *
FROM orders
WHERE price IN (100, 200);
```

Explanation: Converting `OR` conditions to `IN` simplifies the query and often enables MySQL to optimize it similarly. The result is the same, but the `IN` form is clearer and can be better optimized (and still uses any index on `price`).

18. **Inefficient Query:**

```
SELECT e.name, d.*
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

Optimized Query:

```
SELECT e.name, d.name
FROM employees e
JOIN departments d ON e.department_id = d.id;
```

Explanation: Selecting `d.*` fetches all department columns, many of which may be unnecessary. By selecting only `d.name`, we reduce data transfer and processing time, improving performance.

19. **Inefficient Query:**

```
SELECT *
FROM customers
WHERE customer_id = 1 OR customer_id = 2;
```

Optimized Query:

```sql
SELECT *
FROM customers
WHERE customer_id IN (1, 2);
```

Explanation: Using `IN` instead of multiple `OR` conditions is cleaner and allows the optimizer to handle the list efficiently. It also clearly indicates use of the `customer_id` index for those values.

20. **Inefficient Query:**

```sql
INSERT INTO employees VALUES (1, 'Alice', 'HR', 50000);
```

Optimized Query:

```sql
INSERT INTO employees (id, name, department, salary)
VALUES (1, 'Alice', 'HR', 50000);
```

Explanation: Specifying column names is good practice. It does not affect runtime performance, but it makes the query clearer and safer against schema changes.

21. **Inefficient Query:**

```sql
SELECT e.name, SUM(o.total)
FROM employees e
JOIN orders o ON e.id = o.employee_id;
```

Optimized Query:

```sql
SELECT e.name, SUM(o.total)
FROM employees e
JOIN orders o ON e.id = o.employee_id
GROUP BY e.name;
```

Explanation: Adding `GROUP BY e.name` ensures the aggregate `SUM(o.total)` is computed per employee. Without `GROUP BY`, the query is invalid if not selecting `e.name` in an aggregate or list.

22. **Inefficient Query:**

```sql
SELECT name
FROM customers
WHERE LOWER(name) = 'john';
```

Optimized Query:

```
SELECT name
FROM customers
WHERE name = 'john' COLLATE utf8mb4_general_ci;
```

Explanation: Using `LOWER(name)` prevents an index on `name` from being used. Instead, using a case-insensitive collation comparison (or ensuring data is stored in a uniform case) lets MySQL use the index on `name`.

23. **Inefficient Query:**

```
SELECT *
FROM employees
WHERE id = 5;
```

Optimized Query:

```
ALTER TABLE employees ADD PRIMARY KEY (id);
SELECT *
FROM employees
WHERE id = 5;
```

Explanation: If `id` is the primary key, adding a primary key (which creates a unique index) ensures lookups by `id` are very fast. This reduces query time for lookups like `WHERE id = 5` [10].

24. **Inefficient Query:**

```
SELECT *
FROM big_table
LIMIT 1000;
```

Optimized Query:

```
SELECT col1, col2
FROM big_table
LIMIT 1000;
```

Explanation: Even when using `LIMIT`, selecting only the necessary columns (`col1, col2`) instead of `*` can reduce I/O. The `LIMIT` restricts row count, but avoiding `*` further improves performance by reading fewer columns.

25. **Inefficient Query:**

```
SELECT name
FROM employees
WHERE department = 'IT'
  AND (status = 'Active' OR status = 'On Leave');
```

Optimized Query:

```
SELECT name
FROM employees
WHERE department = 'IT'
  AND status IN ('Active', 'On Leave');
```

Explanation: Replacing the `OR` on the same column with `IN` simplifies the condition. MySQL treats this efficiently and can still use an index on `status`.

26. **Inefficient Query:**

```
SELECT SUM(salary)
FROM employees
WHERE department = 'Sales';
```

Optimized Query:

```
ALTER TABLE employees ADD INDEX idx_department (department);
SELECT SUM(salary)
FROM employees
WHERE department = 'Sales';
```

Explanation: Adding an index on `department` lets MySQL quickly find all sales employees. Summing over those rows then only processes the necessary subset, rather than scanning every employee.

27. **Inefficient Query:**

```
SELECT p.name
FROM products p
JOIN categories c ON p.cat_id = c.id
WHERE c.name = 'Electronics';
```

Optimized Query:

```sql
ALTER TABLE categories ADD UNIQUE INDEX idx_cat_name (name);
SELECT p.name
FROM products p
JOIN categories c ON p.cat_id = c.id
WHERE c.name = 'Electronics';
```

Explanation: Adding an index on `categories.name` allows the join condition and filter to use it. MySQL can find the category row by name quickly, then join to products. This avoids scanning the categories table for the row where `name='Electronics'`.

28. **Inefficient Query:**

```sql
SELECT COUNT(*)
FROM sessions
WHERE user_id IS NULL;
```

Optimized Query:

```sql
ALTER TABLE sessions ADD INDEX idx_user_id (user_id);
SELECT COUNT(*)
FROM sessions
WHERE user_id IS NULL;
```

Explanation: Indexing `user_id` can speed up counting rows matching a condition, especially if many sessions exist. The index lets MySQL find the NULL values more efficiently than scanning every row.

29. **Inefficient Query:**

```sql
SELECT *
FROM orders
WHERE order_date >= '2023-01-01'
  AND order_date < '2023-02-01'
ORDER BY order_date;
```

Optimized Query:

```sql
ALTER TABLE orders ADD INDEX idx_order_date (order_date);
SELECT *
FROM orders
WHERE order_date >= '2023-01-01'
```

```
  AND order_date < '2023-02-01'
ORDER BY order_date;
```

Explanation: Indexing `order_date` allows MySQL to both filter and retrieve rows in order directly from the index. This avoids sorting and speeds the query, especially when combined with the date range filter.

30. **Inefficient Query:**

```
SELECT name
FROM customers
WHERE email LIKE '%@gmail.com';
```

Optimized Query:

```
-- Full-text search may be needed for suffix searches
-- Alternatively:
SELECT name
FROM customers
WHERE email LIKE '%@gmail.com'
LIMIT 100;
```

Explanation: A leading wildcard (`%`) prevents index use on `email`. In this case, the best optimization is often to rethink the approach (e.g., maintain a separate column for domain or use full-text search). Here we at least added `LIMIT` to limit rows if applicable.

31. **Inefficient Query:**

```
SELECT *
FROM logs
ORDER BY log_time DESC;
```

Optimized Query:

```
ALTER TABLE logs ADD INDEX idx_log_time (log_time);
SELECT *
FROM logs
ORDER BY log_time DESC;
```

Explanation: Without an index, this sorts all rows by `log_time`. Adding an index means MySQL can scan the index in reverse to get the latest logs first, making the sort instantaneous.

32. **Inefficient Query:**

```sql
SELECT title, author
FROM books
WHERE published_year IN (SELECT year FROM whitelist);
```

Optimized Query:

```sql
SELECT b.title, b.author
FROM books b
JOIN whitelist w ON b.published_year = w.year;
```

Explanation: The subquery can be converted to a join, allowing use of indexes on `published_year` and `year`. Joins like this tend to execute faster than `WHERE IN (subquery)`.

33. **Inefficient Query:**

```sql
SELECT u.id
FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.id
);
```

Optimized Query:

```sql
SELECT DISTINCT u.id
FROM users u
JOIN orders o ON o.user_id = u.id;
```

Explanation: An `INNER JOIN` with `DISTINCT` can achieve the same result (users with orders) and allows MySQL to use indexes on the join. This can be more efficient than a correlated `EXISTS` for large tables.

34. **Inefficient Query:**

```sql
SELECT *
FROM customer_data
WHERE is_active = 1
LIMIT 5000;
```

Optimized Query:

```sql
SELECT id, name, email
FROM customer_data
```

```
WHERE is_active = 1
LIMIT 100;
```

Explanation: First, selecting only needed columns reduces I/O. Second, lowering the `LIMIT` to the actual required number of rows (100 instead of 5000) avoids fetching unnecessary data.

35. **Inefficient Query:**

```
SELECT *
FROM payments
WHERE status = 'Pending' OR status = 'Delayed';
```

Optimized Query:

```
SELECT *
FROM payments
WHERE status IN ('Pending', 'Delayed');
```

Explanation: Using `IN` is equivalent to the `OR` conditions but cleaner. It allows MySQL to use an index on `status` effectively, simplifying the execution plan.

36. **Inefficient Query:**

```
SELECT name
FROM employees
WHERE employee_code = 'ABC123';
```

Optimized Query:

```
ALTER TABLE employees ADD UNIQUE INDEX idx_emp_code (employee_code);
SELECT name
FROM employees
WHERE employee_code = 'ABC123';
```

Explanation: Adding a unique index on `employee_code` ensures lookups by that code are very fast (since it's unique). This speeds up queries that filter by `employee_code`.

37. **Inefficient Query:**

```
SELECT *
FROM visitors;
```

Optimized Query:

```
SELECT id, visit_date, pages_viewed
FROM visitors;
```

Explanation: If only certain columns (e.g., `id, visit_date, pages_viewed`) are needed, specifying them avoids reading all columns, which is more efficient, especially on wide tables.

38. **Inefficient Query:**

```
SELECT student_id, COUNT(*)
FROM enrollments;
```

Optimized Query:

```
SELECT student_id, COUNT(*)
FROM enrollments
GROUP BY student_id;
```

Explanation: The original query lacks `GROUP BY`, so it is invalid for aggregation. Adding `GROUP BY student_id` correctly computes a count per student, allowing the query to run properly.

39. **Inefficient Query:**

```
SELECT *
FROM orders
WHERE discount != 0;
```

Optimized Query:

```
ALTER TABLE orders ADD INDEX idx_discount (discount);
SELECT *
FROM orders
WHERE discount != 0;
```

Explanation: If many orders have no discount, an index on `discount` helps locate the rows with `discount != 0`. Note: inequalities (`!=`) may still scan more, but indexing narrows down the search.

40. **Inefficient Query:**

```sql
SELECT *
FROM employees
WHERE start_date > '2022-01-01'
ORDER BY name;
```

Optimized Query:

```sql
ALTER TABLE employees ADD INDEX idx_start_date (start_date);
SELECT *
FROM employees
WHERE start_date > '2022-01-01'
ORDER BY name;
```

Explanation: Indexing `start_date` helps filter rows in the `WHERE` clause, reducing the dataset. If `name` is also indexed or is the primary key, the `ORDER BY` may use that index. At least, fewer rows need sorting after filtering.

41. **Inefficient Query:**

```sql
SELECT *
FROM products
WHERE price >= 100
  AND price <= 200;
```

Optimized Query:

```sql
ALTER TABLE products ADD INDEX idx_price (price);
SELECT *
FROM products
WHERE price >= 100
  AND price <= 200;
```

Explanation: Adding an index on `price` allows the range condition to use the index efficiently. MySQL can then directly jump to price 100 and scan up to 200, instead of scanning all products.

42. **Inefficient Query:**

```sql
SELECT title
FROM books
WHERE author LIKE 'A%';
```

Optimized Query:

```sql
ALTER TABLE books ADD INDEX idx_author (author);
SELECT title
FROM books
WHERE author LIKE 'A%';
```

Explanation: Because the wildcard is at the end (`'A%'`), MySQL can use an index on `author`. The index greatly speeds up finding all authors starting with 'A'.

43. **Inefficient Query:**

```sql
SELECT email
FROM users
WHERE email IS NOT NULL
  AND id > 1000;
```

Optimized Query:

```sql
ALTER TABLE users ADD INDEX idx_id (id);
SELECT email
FROM users
WHERE email IS NOT NULL
  AND id > 1000;
```

Explanation: Indexing `id` allows MySQL to quickly skip to `id = 1001` and filter from there. If `email` is guaranteed not null beyond that, this avoids scanning the entire table.

44. **Inefficient Query:**

```sql
SELECT name
FROM categories
WHERE description LIKE '%discount%';
```

Optimized Query:

```sql
-- For full-text, but if using LIKE:
ALTER TABLE categories ADD FULLTEXT(description);
SELECT name
FROM categories
WHERE MATCH(description) AGAINST('discount');
```

Explanation: A `%` wildcard at the beginning prevents a regular index use. A full-text index on `description` allows searching for the word 'discount' efficiently.

45. **Inefficient Query:**

```
SELECT COUNT(*)
FROM visits;
```

Optimized Query:

```
-- If COUNT(*) is slow, maintain a counter table or use InnoDB's metadata:
-- For InnoDB, COUNT(*) always scans; instead:
SELECT COUNT(id) FROM visits;
```

Explanation: In InnoDB, `COUNT(*)` is not optimized (it scans rows). Using a secondary table or caching the total count is needed for faster results. (Replacing with `COUNT(id)` still scans; in MyISAM `COUNT(*)` is fast though.)

46. **Inefficient Query:**

```
SELECT *
FROM sessions
WHERE ip_address = '192.168.1.1';
```

Optimized Query:

```
ALTER TABLE sessions ADD INDEX idx_ip (ip_address);
SELECT *
FROM sessions
WHERE ip_address = '192.168.1.1';
```

Explanation: Indexing `ip_address` lets MySQL quickly find sessions with that IP. Without the index, it would scan all sessions.

47. **Inefficient Query:**

```
SELECT *
FROM data
ORDER BY created_at, id;
```

Optimized Query:

```
ALTER TABLE data ADD INDEX idx_created_id (created_at, id);
SELECT *
```

```
FROM data
ORDER BY created_at, id;
```

Explanation: Adding a composite index `(created_at, id)` matches the `ORDER BY`. MySQL can scan the index in order, avoiding a sort, and retrieve rows directly in the desired order.

48. **Inefficient Query:**

```
SELECT col1
FROM t1
WHERE col2 = 'X'
AND col3 = 'Y';
```

Optimized Query:

```
ALTER TABLE t1 ADD INDEX idx_cols (col2, col3);
SELECT col1
FROM t1
WHERE col2 = 'X'
  AND col3 = 'Y';
```

Explanation: A composite index on `(col2, col3)` can be used for both conditions simultaneously, making the lookup much faster than using two separate indexes or a full scan.

49. **Inefficient Query:**

```
SELECT * FROM t
WHERE a = 10 AND (b = 20 OR c = 30);
```

Optimized Query:

```
SELECT * FROM t
WHERE a = 10 AND b = 20
UNION ALL
SELECT * FROM t
WHERE a = 10 AND c = 30;
```

Explanation: Splitting the `OR` into two queries with `UNION ALL` can use indexes on `(a,b)` and `(a,c)` separately. This avoids the inefficiency of `OR` across columns, at the cost of merging results.

50. **Inefficient Query:**

```sql
SELECT * FROM mytable;
```

Optimized Query:

```sql
SELECT column1, column2 FROM mytable;
```

Explanation: Even without a `WHERE`, selecting only needed columns is more efficient. It reduces disk I/O and memory usage. If the table is very large and only some columns are required, always list those instead of `*`.

---

[1] [9] [19] MySQL :: MySQL 8.4 Reference Manual :: 10.2.1 Optimizing SELECT Statements
https://dev.mysql.com/doc/refman/8.4/en/select-optimization.html

[2] [11] [14] [16] [17] [18] 12 SQL Query Optimization Techniques to Follow
https://www.thoughtspot.com/data-trends/data-modeling/optimizing-sql-queries

[3] [21] MySQL performance: INNER JOIN vs. sub-select
https://www.codersrevolution.com/blog/MySQL-performance-INNER-JOIN-vs-subselect

[4] Joins versus Subqueries: Which Is Faster?
https://www.navicat.com/en/company/aboutus/blog/1704-joins-versus-subqueries-which-is-faster.html

[5] MySQL :: MySQL 8.4 Reference Manual :: 10.2.1.19 LIMIT Query Optimization
https://dev.mysql.com/doc/refman/8.4/en/limit-optimization.html

[6] Huge Performance Cost to using SQL Server ORDER BY clause? - Stack Overflow
https://stackoverflow.com/questions/24896360/huge-performance-cost-to-using-sql-server-order-by-clause

[7] MySQL - low performance with WHERE, ORDER BY, LIMIT - Database Administrators Stack Exchange
https://dba.stackexchange.com/questions/326775/mysql-low-performance-with-where-order-by-limit

[8] [20] MySQL :: MySQL 8.4 Reference Manual :: 10.2.1.23 Avoiding Full Table Scans
https://dev.mysql.com/doc/refman/8.4/en/table-scan-avoidance.html

[10] MySQL PRIMARY KEY Indexes: Usage & Examples
https://www.datacamp.com/doc/mysql/mysql-primary-key

[12] In what way does denormalization improve database performance? - Stack Overflow
https://stackoverflow.com/questions/2349270/in-what-way-does-denormalization-improve-database-performance

[13] sql - Why do functions on columns prevent the use of indexes? - Stack Overflow
https://stackoverflow.com/questions/37927069/why-do-functions-on-columns-prevent-the-use-of-indexes

[15] MySQL :: MySQL 8.4 Reference Manual :: 10.2.1.17 GROUP BY Optimization
https://dev.mysql.com/doc/refman/8.4/en/group-by-optimization.html